

COMP 2322 Computer Networking

Multi-thread Web Server

Project Report

Name: Meng Zihan
Student ID: 24110285D
Email: 24110285d@connect.polyu.hk
Date: April 20, 2026

Contents

1	Overview	2
1.1	Project Structure	2
2	Design Highlights	2
2.1	Robust HTTP Request Reading	2
2.2	Accurate Cache Validation with Timestamp Comparison	2
2.3	Robust Path Resolution and Traversal Protection	2
2.4	Thread-Safe Logging with Locking	3
2.5	Persistent Connection Management	3
2.6	Dynamic Status Dashboard with XSS Protection	3
2.7	Proper Content-Length for UTF-8 Responses	3
3	Implementation - Three Stages	3
3.1	Stage 1: Basic Socket Server	3
3.2	Stage 2: File Serving	3
3.3	Stage 3: Multi-threading & Advanced Features	4
4	Additional Features	4
4.1	Cache Validation (304)	4
4.2	Keep-Alive	4
4.3	Status Dashboard	4
5	Test Results	5
5.1	Test 1: Server Startup	5
5.2	Test 2: Stage 1 Request Display	5
5.3	Test 3: GET Text File	5
5.4	Test 4: GET Image File	6
5.5	Test 5: HEAD Method	6
5.6	Test 6: 404 Not Found	6
5.7	Test 7: 403 Forbidden	7
5.8	Test 8: 400 Bad Request	7
5.9	Test 9-10: Last-Modified and 304 Not Modified	7
5.10	Test 11-12: Connection Keep-Alive and Close	8
5.11	Test 13: Multi-threading Concurrent Load	9
5.12	Test 14: Access Log	9
5.13	Test 15: Status Dashboard	10
6	Conclusion	11

1 Overview

This project implements a multi-threaded web server from scratch using Python socket programming. Server runs on port **8848**.

1.1 Project Structure

```
project/
  logs/
    server.log
  report/
    Screenshots/
      01_server_startup.png
      ...
      15_status_dashboard.png
    NETWORK_PROJECT_REPORT.pdf
    reports.tex
  webs/
    index.html
    test.jpg
    test.txt
  Cell_tests.ipynb
  README.md
  server.py
  status_dashboard.html
```

2 Design Highlights

2.1 Robust HTTP Request Reading

Unlike simple implementations that assume the entire request arrives in a single `recv()` call, this server implements a loop that reads until the complete header section is received (indicated by `\r\n\r\n`). This prevents failures when requests are split across multiple TCP packets.

2.2 Accurate Cache Validation with Timestamp Comparison

The server correctly implements HTTP conditional requests. It parses the `If-Modified-Since` header using `email.utils.parsedate_to_datetime()` and compares timestamps at second granularity, avoiding the common mistake of string-based comparison which is brittle and non-standard.

2.3 Robust Path Resolution and Traversal Protection

Path handling uses `urllib.parse.unquote()` to decode URL-encoded characters, then validates that the resolved absolute path stays within the web root using `os.path.commonpath()`. This two-layer defense prevents directory traversal attacks even when attackers use encoded sequences like `%2e%2e%2f`.

2.4 Thread-Safe Logging with Locking

A threading lock protects log file writes, ensuring that concurrent requests do not produce interleaved or corrupted log entries. Each log entry follows a pipe-separated format: `ip | timestamp | method path | status_code`.

2.5 Persistent Connection Management

The server implements HTTP/1.1 persistent connections by default, keeping sockets open for multiple requests until the client sends `Connection: close`. The `should_keep_alive()` function correctly handles both HTTP/1.0 and HTTP/1.1 conventions.

2.6 Dynamic Status Dashboard with XSS Protection

The `/status` endpoint generates a real-time HTML dashboard showing uptime, active threads, request statistics, and recent logs. All dynamic content (file paths, log entries) is escaped using `html.escape()` to prevent injection attacks.

2.7 Proper Content-Length for UTF-8 Responses

All responses compute `Content-Length` by encoding the body to UTF-8 bytes first, then taking the byte length. This ensures correctness for non-ASCII characters.

3 Implementation - Three Stages

3.1 Stage 1: Basic Socket Server

Simple TCP server that accepts connections and returns a fixed response.

Listing 1: Basic Server Loop

```
1 def main():
2     server_socket = socket.socket(socket.AF_INET, socket.
        SOCK_STREAM)
3     server_socket.bind(('127.0.0.1', 8848))
4     server_socket.listen(5)
5     while True:
6         client_socket, addr = server_socket.accept()
7         data = client_socket.recv(4096)
8         client_socket.send(b"HTTP/1.1 200 OK\r\n\r\nHello")
9         client_socket.close()
```

3.2 Stage 2: File Serving

Added file reading, MIME types, and status codes (200, 404, 403).

Listing 2: File Serving

```
1 file_path = os.path.join('webs', path.lstrip('/'))
2 if os.path.exists(file_path):
3     with open(file_path, 'rb') as f:
```

```
4         content = f.read()
5         send_response(200, content)
6     else:
7         send_response(404, "Not Found")
```

3.3 Stage 3: Multi-threading & Advanced Features

- Each request handled in separate thread
- HEAD method support
- Logging to logs/server.log

Listing 3: Multi-threading

```
1 threading.Thread(target=handle_client, args=(client_socket, addr),
    , daemon=True).start()
```

4 Additional Features

4.1 Cache Validation (304)

Server returns Last-Modified header. If client sends If-Modified-Since and file unchanged, returns 304.

4.2 Keep-Alive

Same TCP connection reused for multiple requests. Loop continues until Connection: close.

4.3 Status Dashboard

Visit <http://127.0.0.1:8848/status> to see:

- Server uptime
- Active threads
- Total requests
- Recent logs
- Status code distribution

5 Test Results

5.1 Test 1: Server Startup

Command: `python server.py`

Expected: Server starts on port 8848, creates webs/ and logs/ directories.

```
PS C:\Users\LENOVO\Desktop\network_project> python server.py
[Info] Created logs/ directory
Server started: http://127.0.0.1:8848
Document root: webs/
Log file: logs/server.log
Status page: http://127.0.0.1:8848/status
Waiting for connections...
```

Figure 1: Server startup

5.2 Test 2: Stage 1 Request Display

Command: Client connects, server displays raw HTTP request.

Expected: Server console shows received HTTP request.

```
[New connection] From: ('127.0.0.1', 61496)
[Active threads] 1
[Request] GET /
[New connection] From: ('127.0.0.1', 55385)
[Active threads] 2
```

Figure 2: Stage 1 - Raw request display

5.3 Test 3: GET Text File

Command: `curl http://127.0.0.1:8848/index.html`

Expected: HTTP 200 OK, returns index.html content.

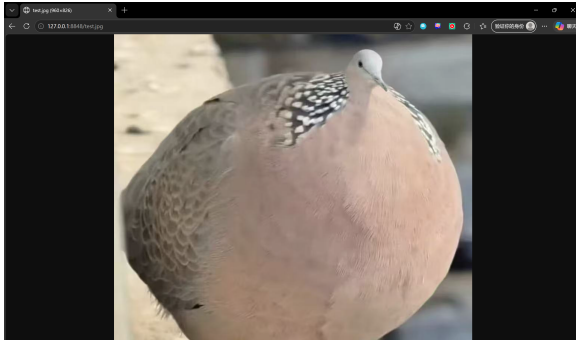
```
C:\Users\LENOVO>curl http://127.0.0.1:8848/index.html
<html>
<body>
  <h1>There will be BIRDS...</h1>
  <p>Gu~Gu~Gu~</p>
</body>
</html>
```

Figure 3: GET text file - 200 OK

5.4 Test 4: GET Image File

Command: `curl -I http://127.0.0.1:8848/test.jpg`

Expected: HTTP 200 OK, Content-Type: image/jpeg. Browser displays image.



```
C:\Users\LENOVO>curl -I http://127.0.0.1:8848/test.jpg
HTTP/1.1 200 OK
Content-Type: image/jpeg
Content-Length: 46161
Last-Modified: Mon, 20 Apr 2026 12:38:06 GMT
Connection: keep-alive
```

Figure 5: curl -I: 200 OK, image/jpeg

Figure 4: Browser displaying image (Spotted Dove)

5.5 Test 5: HEAD Method

Command: `curl -I http://127.0.0.1:8848/index.html`

Expected: HTTP 200 OK with headers only, no message body.

```
C:\Users\LENOVO>curl -I http://127.0.0.1:8848/index.html
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 87
Last-Modified: Mon, 20 Apr 2026 14:32:05 GMT
Connection: keep-alive
```

Figure 6: HEAD method - headers only

5.6 Test 6: 404 Not Found

Command: `curl http://127.0.0.1:8848/nonexistent.html`

Expected: HTTP 404 Not Found with error page.

```
C:\Users\LENOVO>curl http://127.0.0.1:8848/nonexistent.html

<html>
<head><title>404 Not Found</title></head>
<body>
  <h1>404 Not Found</h1>
  <p>File '/nonexistent.html' not found</p>
  <hr>
  <p>Multi-Thread Web Server | <a href="/status">Server Status</a></p>
</body>
</html>
```

Figure 7: 404 Not Found

5.7 Test 7: 403 Forbidden

Command: `curl --path-as-is http://127.0.0.1:8848/../server.py`

Expected: HTTP 403 Forbidden (path traversal protection).

```
C:\Users\LENOVO>curl --path-as-is http://127.0.0.1:8848/../server.py

<html>
<head><title>403 Forbidden</title></head>
<body>
  <h1>403 Forbidden</h1>
  <p>Access denied</p>
  <hr>
  <p>Multi-Thread Web Server | <a href="/status">Server Status</a></p>
</body>
</html>
```

Figure 8: 403 Forbidden - path traversal protection

5.8 Test 8: 400 Bad Request

Command: Python socket (see Jupyter Notebook Cell 7)

Expected: HTTP 400 Bad Request.

```
Cell 7

Purpose: Send an invalid raw socket request and verify the 400 Bad Request response.

print("--- Test Item: 400 Bad Request ---")
try:
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.connect(("127.0.0.1", PORT))
    bad_request = b"HELLO THIS IS NOT HTTP\r\n\r\n"
    s.send(bad_request)
    raw_response = s.recv(1024).decode("utf-8", errors="ignore")
    s.close()

    status_line = raw_response.split("\r\n")[0]
    print("Sent invalid request: HELLO THIS IS NOT HTTP")
    print(f"Server response status line: {status_line}")
except Exception as e:
    print(f"Connection failed: {e}")

✓ 0.0s

--- Test Item: 400 Bad Request ---
Sent invalid request: HELLO THIS IS NOT HTTP
Server response status line: HTTP/1.1 400 Bad Request
```

Figure 9: 400 Bad Request (Cell 7 output)

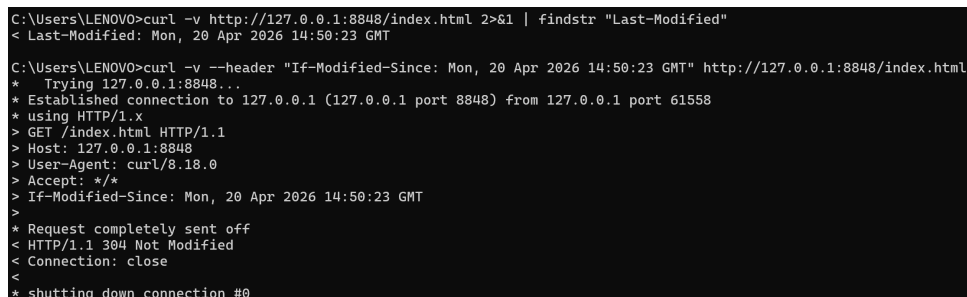
5.9 Test 9-10: Last-Modified and 304 Not Modified

Step 1: Get Last-Modified header

```
curl -v http://127.0.0.1:8848/index.html 2>&1 | findstr "Last-Modified"
< Last-Modified: Mon, 20 Apr 2026 14:50:23 GMT
```

Step 2: Conditional request with If-Modified-Since

```
curl -v --header "If-Modified-Since: Mon, 20 Apr 2026 14:50:23 GMT" http://127.0.0.1:
< HTTP/1.1 304 Not Modified
< Connection: close
```



```
C:\Users\LENOVO>curl -v http://127.0.0.1:8848/index.html 2>&1 | findstr "Last-Modified"
< Last-Modified: Mon, 20 Apr 2026 14:50:23 GMT
C:\Users\LENOVO>curl -v --header "If-Modified-Since: Mon, 20 Apr 2026 14:50:23 GMT" http://127.0.0.1:8848/index.html
* Trying 127.0.0.1:8848...
* Established connection to 127.0.0.1 (127.0.0.1 port 8848) from 127.0.0.1 port 61558
* using HTTP/1.x
> GET /index.html HTTP/1.1
> Host: 127.0.0.1:8848
> User-Agent: curl/8.18.0
> Accept: */*
> If-Modified-Since: Mon, 20 Apr 2026 14:50:23 GMT
>
* Request completely sent off
< HTTP/1.1 304 Not Modified
< Connection: close
<
* shutting down connection #0
```

Figure 10: Last-Modified header and 304 Not Modified response

5.10 Test 11-12: Connection Keep-Alive and Close

Source: Jupyter Notebook Cell 9

Test A: Keep-Alive - Connection header handling

Sent header: {'Connection': 'keep-alive'}

Returned Connection header: keep-alive

Test B: Connection Close

Sent header: {'Connection': 'close'}

Returned Connection header: close

Cell 9

Purpose: Test how the server handles the `Connection` header for both `keep-alive` and `close`.

```
print("--- Test Item: Connection header ---")
url = f"{BASE_URL}/index.html"

headers_keep = {"Connection": "keep-alive"}
r_keep = requests.get(url, headers=headers_keep)
print(f"Sent header: {headers_keep}")
print(f"Returned Connection header: {r_keep.headers.get('Connection')}")
print()

headers_close = {"Connection": "close"}
r_close = requests.get(url, headers=headers_close)
print(f"Sent header: {headers_close}")
print(f"Returned Connection header: {r_close.headers.get('Connection')}")
```

```
--- Test Item: Connection header ---
Sent header: {'Connection': 'keep-alive'}
Returned Connection header: keep-alive

Sent header: {'Connection': 'close'}
Returned Connection header: close
```

Figure 11: Connection header handling - keep-alive and close (Cell 9)

5.11 Test 13: Multi-threading Concurrent Load

Command: 10 simultaneous GET requests via Python threading.

Expected: All 10 requests return 200 OK.

```
C:\Users\LENOVO>for /l %i in (1,1,10) do start /b curl http://127.0.0.1:8848/index.html
C:\Users\LENOVO>start /b curl http://127.0.0.1:8848/index.html
C:\Users\LENOVO>start /b curl http://127.0.0.1:8848/index.html
C:\Users\LENOVO>start /b curl http://127.0.0.1:8848/index.html
<html>
<body>
  <h1>There will be BIRDS...</h1>
  <p>Gu~Gu~Gu~</p>
</body>
</html>
C:\Users\LENOVO>start /b curl http://127.0.0.1:8848/index.html
<html>
<body>
  <h1>There will be BIRDS...</h1>
  <p>Gu~Gu~Gu~</p>
</body>
</html>
C:\Users\LENOVO>start /b curl http://127.0.0.1:8848/index.html
<html>
<body>
  <h1>There will be BIRDS...</h1>
  <p>Gu~Gu~Gu~</p>
</body>
</html>
C:\Users\LENOVO>start /b curl http://127.0.0.1:8848/index.html
<html>
<body>
  <h1>There will be BIRDS...</h1>
  <p>Gu~Gu~Gu~</p>
</body>
</html>
C:\Users\LENOVO>start /b curl http://127.0.0.1:8848/index.html
<html>
<body>
  <h1>There will be BIRDS...</h1>
```

Figure 12: 10 concurrent requests - all successful

5.12 Test 14: Access Log

Command: type logs\server.log (Windows) or cat logs/server.log

Expected: Each request recorded with IP, timestamp, request, status code.

```

C:\Users\LENOVO\Desktop\network_project>type logs\server.log
127.0.0.1 | 2026-04-10 23:23:42 | GET /index.html | 200
127.0.0.1 | 2026-04-14 22:36:11 | GET /index.html | 200
127.0.0.1 | 2026-04-14 22:36:11 | GET /favicon.ico | 404
127.0.0.1 | 2026-04-14 22:36:45 | HEAD /index.html | 200
127.0.0.1 | 2026-04-14 22:36:55 | HEAD /index.html | 200
127.0.0.1 | 2026-04-20 20:20:51 | GET /status | 200
127.0.0.1 | 2026-04-20 20:21:22 | GET /status | 200
127.0.0.1 | 2026-04-20 20:21:22 | GET /favicon.ico | 404
127.0.0.1 | 2026-04-20 20:21:29 | GET /status | 200
127.0.0.1 | 2026-04-20 20:21:34 | GET /status | 200
127.0.0.1 | 2026-04-20 20:38:55 | GET /test.jpg | 200
127.0.0.1 | 2026-04-20 20:38:55 | GET /favicon.ico | 404
127.0.0.1 | 2026-04-20 21:07:21 | GET /index.html | 200
127.0.0.1 | 2026-04-20 21:07:33 | GET /test.png | 404
127.0.0.1 | 2026-04-20 21:07:56 | GET /test.jpg | 200
127.0.0.1 | 2026-04-20 21:08:04 | HEAD /index.html | 200
127.0.0.1 | 2026-04-20 21:08:17 | GET /this_file_does_not_exist.html | 404
127.0.0.1 | 2026-04-20 21:08:26 | GET ../server.py | 403
127.0.0.1 | 2026-04-20 21:08:39 | HELLO THIS | 404
127.0.0.1 | 2026-04-20 21:08:54 | GET /index.html | 200
127.0.0.1 | 2026-04-20 21:08:54 | GET /index.html | 200
127.0.0.1 | 2026-04-20 21:09:00 | GET /index.html | 200
127.0.0.1 | 2026-04-20 21:09:00 | GET /index.html | 200
127.0.0.1 | 2026-04-20 21:09:03 | GET /index.html | 200
127.0.0.1 | 2026-04-20 21:09:03 | GET /index.html | 200
127.0.0.1 | 2026-04-20 21:09:03 | GET /index.html | 200
127.0.0.1 | 2026-04-20 21:09:03 | GET /index.html | 200
127.0.0.1 | 2026-04-20 21:09:03 | GET /index.html | 200
127.0.0.1 | 2026-04-20 21:09:03 | GET /index.html | 200
127.0.0.1 | 2026-04-20 21:09:46 | GET /index.html | 200
127.0.0.1 | 2026-04-20 21:09:53 | GET /test.jpg | 200
127.0.0.1 | 2026-04-20 21:20:39 | GET /index.html | 200
127.0.0.1 | 2026-04-20 21:20:39 | GET /test.jpg | 200
127.0.0.1 | 2026-04-20 21:20:39 | HEAD /index.html | 200
127.0.0.1 | 2026-04-20 21:20:39 | GET /this_file_does_not_exist.html | 404
127.0.0.1 | 2026-04-20 21:20:39 | GET ../server.py | 403
127.0.0.1 | 2026-04-20 21:20:39 | HELLO THIS | 404
127.0.0.1 | 2026-04-20 21:20:39 | GET /index.html | 200
127.0.0.1 | 2026-04-20 21:20:39 | GET /index.html | 200
127.0.0.1 | 2026-04-20 21:20:39 | GET /index.html | 200
127.0.0.1 | 2026-04-20 21:20:39 | GET /index.html | 200
127.0.0.1 | 2026-04-20 21:20:39 | GET /index.html | 200
127.0.0.1 | 2026-04-20 21:20:39 | GET /index.html | 200
127.0.0.1 | 2026-04-20 21:20:39 | GET /index.html | 200

```

Figure 13: Access log file contents

5.13 Test 15: Status Dashboard

Method: Visit <http://127.0.0.1:8848/status> in Browser

Expected: HTML dashboard showing uptime, active threads, request count, recent logs.

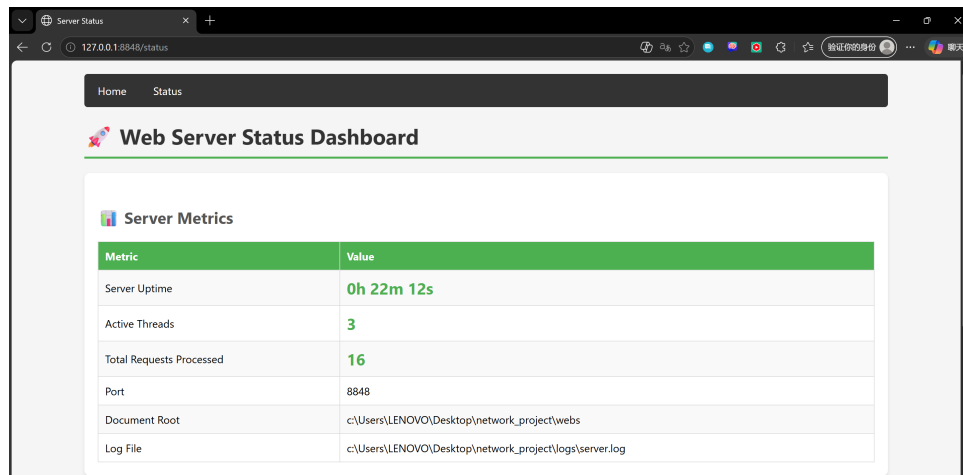


Figure 14: Status dashboard (browser view)

6 Conclusion

Successfully implemented a fully functional multi-threaded web server supporting HTTP/1.1 features including persistent connections, cache validation, and concurrent request handling.